

BUILD PLAN · INDEPENDENT PHARMACY AUTOMATION · v1.1

AI Pharmacy Technician

Automating prescription data entry & pharmacist verification

Strategy, technical approach, and named task assignments for the founding team: **Joel, Ananth, and Vibhas** (engineering), **Matt** (PharmD, clinical lead & idea originator), and **Anjali** (pharmacist on staff at the pilot site). Pilot pharmacy runs **Liberty Software (RxQ)**. Phase-one goal: a working, pharmacist-supervised data-entry assistant proven in one real pharmacy — not a broad platform.

THE WEDGE	THE PILOT	THE APPROACH
Prescription intake → structured extraction → drafted verification queue. The highest-volume, most error-prone task in every pharmacy, every day.	One independent pharmacy on Liberty / RxQ . Owner on board; Anjali works there . Real scripts, real feedback loop.	API-first . Liberty exposes a PIN-based API. No screen-scraping. Pharmacist-in-the-loop on every fill.

What changed in v1.1: named owners for every task; v1 scope now leans on Liberty's built-in clinical/DUR engine instead of building our own interaction checker (NLM's free interaction API was discontinued in Jan 2024, so standalone interaction checking becomes a deliberate Phase-4 build-vs-buy decision); added the HIPAA business-associate agreement with the pilot pharmacy and a state-regulation check as explicit Week-1 tasks.

1 · Where we fit in the market

The space has real, funded competitors — which proves demand. Our job is a sharp wedge, not a broad suite. Here is who's out there and what it means for us.

PLAYER	WHAT THEY DO	WHAT IT MEANS FOR US
Sonet (PAT)	Pure computer-vision agent that drives whatever PMS is on screen. “No APIs, no migration.” Targets prior-auth & billing queues.	Brittle to UI changes; slower per action; the riskiest path for a clinical task. We go API-first instead and keep vision only as a last-resort fallback.
Asepha	Broad suite: data entry, call assistant, BV/PA, 340B, REMS, med rec, med review. Reports ~32% manual-labor cost cut, 4x faster tasks.	Breadth is their weakness for a single-store owner. We win on focus + a simpler, trust-first pitch, then expand once we own the data-entry beachhead.
PMS-native (e.g. Rx30)	Rx30 now ships built-in “Automated Data Entry” that auto-populates eScript fields before staff start the day.	Proves pharmacists want exactly this. But it only exists inside one PMS. The other 6–7 platforms (incl. Liberty) have no equivalent — that gap is our market.
CoverMyMeds / Surescripts	Own the e-prescribing & prior-auth network rails at the industry level (NCPDP SCRIPT).	Don't compete with the rails — ride them. Adopt NCPDP SCRIPT as our data model rather than inventing one.

The differentiation we can actually defend

- **Drug-database grounding, not raw OCR.** Every extracted drug + dose is cross-checked against RxNorm / FDA NDC *before* a human sees it. That turns “the AI thinks it read X” into “X is a real drug at a real dose.” Generic document-AI vendors don't do this; it's our moat.
- **Error-reduction, not just time-saving.** Data entry is where ~1 in 4 dispensing errors originate. “We catch mistakes before verification” is a stronger sell to a pharmacist than “we save minutes.”
- **Show-your-work audit trail.** Per-field source, confidence, and every pharmacist correction logged. This is the trust unlock with the actual buyer (the pharmacist, who holds the liability).

2 · The pilot system: Liberty (RxQ)

Good news that shapes everything: **Liberty has a real, PIN-authenticated API** and a large third-party integration ecosystem (DocStation, delivery apps, eMARs, 340B tools, and more). That moves our pilot from “screen-scraping only” up to “proper API integration possible.” We are not fighting the PMS — we plug into it the same way its existing partners do.

How API access actually works (confirmed)

1	In RxQ, go to the System tab.
2	Open Settings (beta builds: Settings → Utilities), then API Keys .
3	Click Add — Liberty generates a random 7-digit key. Label it (e.g. “PAT”) and Save.
4	Our app uses that key/PIN to authenticate and pull patient + dispensing data (the DocStation integration works exactly this way, incl. historical import).

■ VERIFY EARLY (Week 1 — Joel)

The public docs confirm the API exists and supports reading patient/dispensing data, but they don't fully spell out **write-back** scope (can we push a drafted/entered Rx, or is it read-only today?). **Action:** contact Liberty's developer/support team directly through the pharmacy owner's account to get the full API spec, rate limits, and write-back capabilities in writing before architecture is locked. If write is limited, Anjali confirms the drafted record manually inside RxQ while we read everything else via API — still a big time-saver, and a clean v1.

Integration tiers (how we scale past the pilot later)

TIER	SYSTEMS	METHOD
Tier 1	Liberty, PioneerRx / RedSail family	Documented vendor API — read & (where available) write structured records.
Tier 2	Any NCPDP SCRIPT-capable PMS	Standard SCRIPT transactions (NEWRX, REFREQ...). More durable than any one vendor's API.
Tier 3	Legacy / no-API systems	Narrow, heavily-tested “fill-one-form” automation per PMS. Last resort, not the foundation.

3 · The technical pipeline

Five stages. Each emits a structured, versioned object (not free text), so every step is independently testable and fully auditable.



Stage 1 — Intake

Three input types. **eScripts** arrive as NCPDP SCRIPT XML — already structured, parse directly, near-zero error (the easy 60–70% of volume). **Faxes** and **handwritten scripts** come in as images: deskew / denoise / contrast, then route to extraction (handwriting gets a higher-scrutiny path).

Stage 2 — Extraction

Skip legacy OCR as the primary path. Use a **vision-language model with a strict JSON schema** (patient, drug, strength, form, qty, sig, days supply, refills, prescriber, NPI/DEA, date), a **per-field confidence signal**, and a **normalized bounding box per field** in the same pass (this powers the click-to-source overlay in the UI). Then the key step: **resolve every drug name against RxNorm** — exact/normalized lookup first (findRxcuiByString), scored fuzzy retrieval as fallback (getApproximateMatch) — then RxCUI → NDC via RxNorm/openFDA, intersected with the pharmacy's actual stock from Liberty. Ambiguous reads ("5mg" vs "5ng") resolve against reality, not a guess. Note: real-world eScript codes fail to resolve directly ~25% of the time, so this fallback path runs on *all* input types, not just handwriting.

Stage 3 — Match & Validate (v1 scope decision)

Fuzzy-match patient to the existing Liberty profile; resolve drug to a specific stocked NDC (flag out-of-stock early); run **allergy-vs-profile, duplicate-vs-history, and dose-plausibility** checks *before* the pharmacist looks. **For v1 we deliberately lean on Liberty's built-in clinical/DUR engine as the authoritative check at final verification** — it already covers drug-drug, dose, allergy, morphine-equivalent, pregnancy/age alerts. Our pre-checks are additive, not a replacement. Standalone interaction checking is a Phase-4 build-vs-buy decision (NLM discontinued its free interaction API in Jan 2024; options are a commercial DB license such as Medi-Span/FDB, or NLP over openFDA label text).

Stage 4 — Verification Draft (the product surface)

The pharmacist screen. Side-by-side original image (clickable bounding boxes, both directions) + populated fields. **Green / amber / red tiers** — a composite of model signal + RxNorm score margin + NDC-in-stock + patient-history consistency, never the model's self-reported confidence alone. All-green scripts land in a fast lane (one tap to accept, never zero-touch); any amber/red routes to a review lane with focus landing on the riskiest field. Every correction is logged — compliance trail *and* training signal.

Stage 5 — Write-back

Tiered by integration depth. For Liberty: push the verified record via API where supported; otherwise Anjali confirms inside RxQ while we still automate everything upstream. **Never auto-dispense without the pharmacist checkpoint.**

Tooling decisions

LAYER	CHOICE	WHY
Extraction model	Frontier VLM w/ strong structured output + large context	Hundreds of docs/day, not millions — accuracy beats throughput.
Drug resolution	NLM RxNorm REST (exact → approximate) + openFDA NDC	Free, no license needed, authoritative. Hit live endpoints for the pilot; mirror to Postgres later.
Clinical / DUR (v1)	Liberty's built-in clinical engine	Already authoritative at verification; lets us ship the pilot without a DUR build.
Compliance	BAA-covered model endpoint + BAA with the pharmacy	We process PHI on the pharmacy's behalf — we're their business associate under HIPAA.
Stack	Next.js + TypeScript, Prisma, PostgreSQL	Team's existing strength — fast to build, easy to maintain. Polling first; websockets only if needed.

4 · Who does what

Five people, clear lanes. Matt and Anjali are not “advisors” — they are the clinical safety function and the source of ground-truth labels; final clinical judgment calls sit with Matt as the PharmD. The Ananth/Vibhas lanes are swappable based on who’s stronger on ML-pipeline vs product/frontend work — decide at the Week-1 kickoff and stick with it.

JOEL — Lead Engineer · Integration & Infra

Liberty API + orchestration + compliance

Owns: the connector layer, infrastructure, and end-to-end glue

- Own the Liberty relationship: full API spec, keys, rate limits, write-back scope **in writing** (Week 1, via the owner’s account).
- Execute the HIPAA BAAs: pharmacy ↔ us (we are their business associate) and us ↔ model provider.
- Build the Liberty connector: PIN auth, patient + dispensing + inventory pulls, historical import.
- Draft + version the canonical prescription JSON schema (NCPDP-SCRIPT-flavored) all workstreams share.
- Infra: repo, CI, Postgres, secrets management, BAA-covered model endpoint, append-only audit-log table.
- Run the weekly pilot review; coordinate the owner, Matt, and the team.

ANANTH — Extraction & AI Pipeline

Stages 1–2 + drug grounding

Owns: turning scripts into validated, structured data

- eScript (NCPDP SCRIPT XML) parser → canonical schema.
- Image preprocessing for faxes/handwriting (deskew, denoise, contrast).
- VLM extraction: strict JSON schema, per-field confidence **and normalized bounding box** in one pass.
- Drug resolution: findRxcuiByString (exact/normalized) → getApproximateMatch fallback with score margins; RxCUI → NDC via RxNorm/openFDA; intersect with Liberty stock.
- Composite tier scorer (model signal + score margin + stock + history) using Matt’s thresholds.
- Eval harness against the pharmacist-labeled gold set; track calibration from correction data.

VIBHAS — Product & Verification UI

Stages 3–4 — the pharmacist-facing app

Owns: the verification surface the pilot lives in

- Next.js + Prisma app: fast lane / review lane queue, Postgres-backed, polling first (TanStack Query).
- Side-by-side viewer: script image / PDF (pdf.js) + form fields, clickable bounding-box overlay both directions.
- Tier-driven UX: green pre-checked, amber requires acknowledgment, red blocks the fast path; focus auto-lands on the riskiest field.
- Patient match + allergy/duplicate-vs-history pre-checks from Liberty profile data (Liberty’s own DUR stays authoritative at final verify).
- Correction-logging pipeline → audit trail + Ananth’s calibration data.
- Metrics dashboard: seconds-per-script, auto-accept rate, corrections by field, flags caught.

MATT — Clinical Lead & Product Owner*PharmD (finishing this year) · idea originator***Owens:** what “correct” and “safe” mean; the pilot protocol

- Define the field-level correctness rubric and the green/amber/red thresholds (with Ananth).
- Author the safety guardrails: what the system may **never** do without explicit pharmacist action.
- Design the gold-standard labeling protocol (Anjali collects; Matt adjudicates disputes).
- Own the shadow-mode → assist-mode pilot protocol and the success metrics.
- Check state pharmacy regulations (Michigan): data-entry/verification rules, tech ratios, anything touching AI-assisted entry — flag constraints before they surprise us.
- Phase-4 build-vs-buy on standalone interaction checking (commercial DB vs NLP over FDA labels).

ANJALI — Embedded Pilot Pharmacist*On staff at the pilot pharmacy***Owens:** ground truth from the real pharmacy (scoped around her work + study schedule)

- Week 1 baseline: time ~20 real scripts through today’s data-entry + verification flow in RxQ.
- Map the actual RxQ workflow step-by-step, with screenshots of each data-entry screen for the devs.
- With the owner: generate the API key (System → Settings → API Keys) and gather sign-offs.
- Label prescriptions for the gold set per Matt’s protocol (incl. messy faxes and handwriting).
- Daily-use the tool during pilot shifts; log every miss and every false flag.
- Collect edge cases as they occur: controlled substances, compounds, insurance oddities.

5 · Phased plan (first ~12 weeks)

Sequenced so the riskiest unknowns (API write scope; extraction accuracy on real scripts) are retired first.

PHASE	WEEKS	GOAL	DONE WHEN...
0 · Foundations	1–2	De-risk + scaffold. Liberty API spec confirmed; BAAs signed; schema + infra in place; baseline + workflow mapped. <i>Lead: Joel + Anjali.</i>	We can read live patient/dispensing data from the pilot via API, into our schema.
1 · Extraction	3–5	eScript parser + VLM extraction + RxNorm/NDC grounding, scored against the pharmacist-labeled set. <i>Lead: Ananth.</i>	Structured output hits target accuracy on eScripts; tiers are calibrated on faxes.
2 · Verify UI	5–8	Matching + pre-checks + the verification screen with correction logging. <i>Lead: Vibhas.</i>	A pharmacist can review & correct a drafted script end-to-end in the UI.
3 · Pilot	8–12	Run live (shadow first, then assisting) in the pharmacy. Measure, fix, calibrate weekly. <i>Lead: Matt + Anjali.</i>	Measured time saved + flags caught vs. baseline; the pharmacists trust the queue.

Operating rhythm

- **Weekly pilot review** (Joel runs, Matt + Anjali report): what the AI missed, what it flagged correctly, where it saved time. Every correction becomes a test case.
- **Shadow mode before assist mode.** Early on, the AI drafts but changes nothing in RxQ — pharmacists compare against what they'd have typed. Trust is earned before the tool touches the workflow.
- **One metric above all:** median pharmacist seconds-per-script at verification, vs. the Week-1 baseline Anjali measures.

DEFINITION OF A SUCCESSFUL PILOT

The pharmacist spends meaningfully less time per prescription at verification **and** the system catches real errors/flags before they reach the bench — with zero unsupervised dispensing and a complete audit trail. If we hit that in one pharmacy, we have a reference customer, a labeled dataset, and a repeatable Liberty integration to take to the next store.

Immediate next actions (this week)

OWNER	ACTION
Joel	Through the owner's account, request Liberty's full API documentation + confirm write-back scope, rate limits, and any data agreement needed. Start both BAAs (pharmacy ↔ us; us ↔ model provider).
Joel + Anjali	Generate an API key in RxQ (System → Settings → API Keys) and confirm we can pull live dispensing data into a test DB.
Anjali	Time the current data-entry + verification workflow for ~20 real scripts and capture RxQ workflow screenshots — this is the baseline everything is measured against.
Matt	Draft the field-level correctness rubric + initial tier thresholds; start the labeling protocol; begin the Michigan board-of-pharmacy rules check.
Ananth	VLM extraction spike on 10–20 sample scripts (eScript + fax + handwritten) and an RxNorm resolution spike (exact → approximate fallback) to gauge starting accuracy.
Vibhas	Wireframe the verification screen (two-lane queue, bounding-box overlay, tier colors); scaffold the Next.js app + queue table schema.
All	Review + sign off the canonical prescription JSON schema (Joel drafts) and lock the Ananth/Vibhas lane split at the Week-1 kickoff.

This plan is intentionally narrow. Prior-auth, refill outreach, call automation, standalone DUR, and the other PMS integrations are real expansions — but only after we own data entry in one pharmacy and have the trust, the dataset, and the reference customer to earn the next one.